



Build **fast**. Build **right**.

Composable framework for building advanced applications



Romuald Brillout

- Creator of Vike and Telefunc (tRPC alternative)
- Half French/German, grew up in Paris
- Computer Science at KIT (Karlsruhe, Germany)
- Lives in Munich

Intro



History



Hooks



Extensions



Use Case



DX



Business model



Future



Conclusion





vite-plugin-ssr

Like Next.js/Nuxt but as do-one-thing-do-it-well Vite plugin.

- Started vite-plugin-ssr in 2021
 - Vite's early days (before its meteoric rise)
 - Idea: Powerful low-level hooks, for full control over integration
- Renamed to Vike
 - For "street cred"
 - Extensions: vike-react / vike-vue / vike-solid (and more)

```
// pages/hello/+Page.jsx
// Environment: client and/or server

export function Page(pageContext) {
  const confName = "c't <webdev>"
  return <>
    <h1>Hello {confName}</h1>
  </>
}
```

Intro
○

History
○

Hooks
●○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

```
// pages/+onRenderHtml.js
// Environment: server

import ReactDOMServer from 'react-dom/server'

export function onRenderHtml(pageContext) {
  const { Page } = pageContext
  const pageHtml = ReactDOMServer.renderToString(<Page/>)
  return `
    <html>
      <body>
        <div id="root">${pageHtml}</div>
      </body>
    </html>`
}
```

```
// pages/+onRenderClient.js
// Environment: client

import ReactDOMClient from 'react-dom/client'

export function onRenderClient(pageContext) {
  const { Page } = pageContext
  ReactDOMClient.hydrateRoot(
    document.getElementById('root'),
    <Page/>
  )
}
```

```
// Doesn't depend on Node.js (e.g. can be used in Cloudflare Workers)
import { renderPage } from 'vike/server'

// Any server: Express.js, Cloudflare Worker, AWS Lambda Function, Fastify, Hono, Nitro, ...
server.addMiddleware({
  method: 'GET',
  route: '*', // catch-all
  async handler(request) {
    const pageContextInit = { urlOriginal: request.url }
    const pageContext = await renderPage(pageContextInit)
    // `body` is the HTML of the page with a route matching pageContextInit.urlOriginal
    const { body, statusCode, headers } = pageContext.httpResponse
    const response = { body, statusCode, headers }
    return response
  }
})
```

```
// pages/+onCreateGlobalContext.server.js
// Environment: server
export async onCreateGlobalContext() {
  const dataFromAPI = await fetchDataFromAPI()
  const someGlobalData = select(dataFromAPI)
  globalContext.someGlobalData = someGlobalData
}
```

```
// pages/+config.js
export default {
  passToClient: 'someGlobalData'
}
```

```
// Anywhere (client,server)
import { getGlobalContext } from 'vike'
const { someGlobalData } = await getGlobalContext()
```

- `+data.js`
 - Data fetching
- `+onCreateGlobalContext.client.js`
 - Client-side initialization (e.g. authentication)
 - Init store (e.g. Redux or Zustand)
- `+onCreatePageContext.{client,server}.js`
- `+guard.js`
 - Authorization
- `+onPageTransition{Start,End}.js`
 - Full control over page transition animation
- `+onBeforeRoute.js`
 - Full control over i18n integration
- ...

Extensions

- `vike-react`
- `vike-react-rsc`  ⇒ Server Components
- `vike-react-redux`
- `vike-react-zustand`
- `vike-react-query`
- `vike-react-apollo`
- ...

`vike-photon` ⇒ server & deployment

More: `vike-vue`, `vike-vue-pinia`, `vike-vue-query`, `vike-solid`, `vike-angular`, ...

Use case

Burda GmbH owns [focus.de](#), [chip.de](#), [bunte.de](#), ...

- **Before:** fragmented stacks
 - ⇒ cannot improve all websites at once
- **Now:** Internal company framework powering all websites
 - ⇒ Software Architects controls architecture of all websites
 - ⇒ App code: product developers
- **Vike core**, without any Vike extension
 - ⇒ full control over architecture + rock-solid foundation
- `vike-react` is ~1k LoC ⇒ easy to build your own framework

SPA/SSR/SSG

SPA, SSG, SSR, HTML streaming.

⇒ first-class support for all — also SPA

- `+ssr` **boolean** ⇒ toggle SSR/SPA on page-by-page basis
- `+prerender` **boolean** ⇒ disable/enable page-by-page
- SSG redirects
- 🚧 SSG dynamic routes (aka SPA fallback)
- `+data.client.js` ⇒ Data-fetching 100% client-side
- `react-streaming` ⇒ progressive rendering (aka partial hydration)
 - React library we created ⇒ fully Vike agnostic

Easily create SPA like the old days.

⇒ surprisingly hard with other frameworks (but shouldn't!)



Remote Functions. Instead of API.

- Optional: you can use REST/GraphQL instead.
- Simplicity.
- Inspired design of SvelteKit & Solid Start.

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○●○○○○○○○

Business model
○

Future
○

Conclusion
○

```
// CreateTodo.telefunc.ts
// Environment: server

// Telefunc makes onNewTodo() remotely callable
// from the browser.
export { onNewTodo }

import { getContext } from 'telefunc'

// Telefunction arguments are automatically validated
// at runtime, so `text` is guaranteed to be a string.
async function onNewTodo(text: string) {
  const { user } = getContext()

  // With an ORM
  await Todo.create({ text, authorId: user.id })

  // With SQL
  await sql(
    'INSERT INTO todo_items VALUES (:text, :authorId)',
    { text, authorId: user.id }
  )
}
```

Server

```
// CreateTodo.tsx
// Environment: client

// CreateTodo.telefunc.ts isn't actually loaded;
// Telefunc transforms it into a thin HTTP client.
import { onNewTodo } from './CreateTodo.telefunc.ts'

async function onClick(form) {
  const text = form.input.value
  // Behind the scenes, Telefunc makes an HTTP request
  // to the server.
  await onNewTodo(text)
}

function CreateTodo() {
  return (
    <form>
      <input input="text"></input>
      <button onClick={onClick}>Add To-Do</button>
    </form>
  )
}
```

Browser

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○●○○○○○

Business model
○

Future
○

Conclusion
○

React Router 🤔

```
// article/delete.ts
// Only a single action per file 🙄
export async function action({ request }) {
  const { article_id } = await request.json()
  await db.article.delete({ article_id })
  return Response.json({ success: true })
}
```

```
// article.tsx
export default function Article() {
  const onClick = async () => {
    const res = await fetch("/article/delete", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ article_id })
    })
    const { success } = await res.json()
    if (!success) alert("Deletion failed")
  }
  return <button onClick={onClick}>Delete</button>
}
```

Vike + Telefunc ✨

```
// article.telefunc.ts
export async function onArticleDelete({ article_id }) {
  await db.article.delete({ article_id })
  return { success: true }
}
```

```
// article.tsx
import { onArticleDelete } from './article.telefunc.ts'
export default function Article() {
  const onClick = async () => {
    const { success } = await onArticleDelete({ article_id })
    if (!success) alert("Deletion failed")
  }
  return <button onClick={onClick}>Delete</button>
}
```

Next.js 🤔

```
// app/article.tsx
// Environment: client
import { revalidatePath } from 'next/cache'

export default function Article() {
  async function onArticleDelete() {
    'use server'
    // Environment: server
    // 🤔 Mixed server and client code inside same file
    // 🤔 Magic passing of props
    await db.article.delete({ article_id })
    // 🤔 Complex cache handling
    revalidatePath('/articles')
  }

  return (
    <button onClick={onArticleDelete}>Delete</button>
  )
}
```

Vike + Telefunc ✨

```
// article.telefunc.ts
// Environment: server
export async function onArticleDelete({ article_id }) {
  await db.article.delete({ article_id })
  return { success: true }
}
```

```
// article.tsx
// Environment: client
import { onArticleDelete } from './article.telefunc.ts'
export default function Article() {
  const onClick = async () => {
    const { success } = await onArticleDelete({ article_id })
    if (!success) alert("Deletion failed")
  }
  return <button onClick={onClick}>Delete</button>
}
```

✅ No caching by default (but you can if you want)

RPC

- Not built-in by design (unlike Remix and Next.js)
- Any RPC tool: Telefunc, tRPC, ...
- Any non-RPC tool: React Query, GraphQL, ...
- Progressive migration from one paradigm to another
 - E.g. progressively migrate from GraphQL to RPC

npm pnpm yarn bun

>_ npm create vite@latest --- --react --tailwindcss --daisyui --authjs --telefunc --hono --drizzle --eslint

 Try me in Stackblitz

Presets

Frontend

Full-stack

Next.js

Nuxt

CMS

Frontend

Frontend Framework (required)



Vike

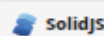
UI Framework (required)



React

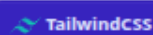


Vue



SolidJS

CSS

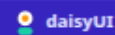


TailwindCSS

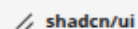


Compiled

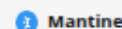
UI Component Libraries



daisyUI



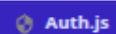
shadcn/ui



Mantine

Data

Auth



Auth.js

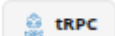


Auth0

Data fetching



Telefunc

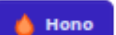


tRPC



ts-rest

Server



Hono



h3

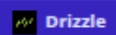


Express

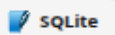


Fastify

Database



Drizzle



SQLite



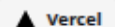
Prisma

Deployment

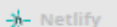
Hosting



Cloudflare



Vercel



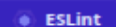
Netlify



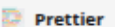
AWS

Utilities

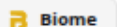
Linter



ESLint

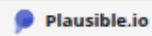


Prettier



Biome

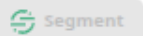
Analytics



Plausible.io



Google Analytics

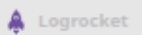


Segment

Error tracking



Sentry



Logrocket

Details

- Bugs quickly fixed (usually under 24h)
- Deployment skew protection
- Different Base URL for client/server
- `>=400` helpful warnings & error messages
 - Minimal in client-side production to save KBs
- Infinite loop guards
- CSP support
- ISR support (via `@photonjs/vercel`)
- Route Functions
- ...

Open Source Pricing

- 100% MIT licensed
- 100% free for engineers
 - No license needed — works like a normal open source tool
- Only larger companies pay
 - They see a pesky toaster in dev: Purchase a license at ...
 - Vike heuristic detects whether app is developed by a team
 - Pay only how much you can/want

It's not a bug, it's a feature.

Future

- Powerful extensions
 - Full-stack extensions, like Django / Ruby on Rails
 - Full-stack easy path
 - Eject ⇒ progressive & cherry-picked control

Our belief: Vike's architecture + powerful extensions is the future.

Why Vike

- ✓ Architecture (flexible core + powerful extensions)
- ✓ Next-gen DX
- ✓ Polished details
- ✓ Vibrant ecosystem
- ✓ Transparent business model

Vike Team

- Rom • Germany, Munich
- Joël • France, Belfort
- Dani • Hungary, Lake Balaton
- Muhammad • Indonesia, Aceh

Questions?

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
●

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Slide

Foo bar

Intro
○

History
○

Hooks
○○○○○○○

Extensions
○

Use Case
○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○